React JS
Interview Questions

Hooks, reconciliation/Fiber internals, patterns, and machine-coding rounds

Compiled from real interview experiences (GrowthGear, qpiAI, CREATE Bytes, PSQUARE) + curated additions

---

Topics covered: JSX & Virtual DOM · hooks (useState/useEffect/useRef/useMemo/useCallback) · custom hooks · HOC vs memo · Context vs Redux Toolkit · Fiber & reconciliation · Suspense/lazy loading · performance · machine coding (pagination, debounced search, infinite scroll)
Difficulty split: Easy → Medium → Hard, each with theory, output-based, and coding questions

Format: Every section is split into **Easy** → **Medium** → **Hard**, and spans theoretical concepts, output-based/tricky code, and practical / machine-coding style problems.

EASY LEVEL

React Fundamentals

1. What is JSX? Why can't browsers run it directly, and what converts it to plain JS?

2. What is the Virtual DOM? Why does React use it instead of updating the real DOM directly?

3. What is the difference between a functional component and a class component?

4. What are props? Are they mutable or immutable inside a child component?

5. What is 'state' in React, and how is it different from props?

Core Hooks

6. What is useState and how does it differ from a regular variable inside a component?

7. What is useEffect used for? What does an empty dependency array [] mean vs no array at all?

8. Why does React require a unique 'key' prop when rendering lists? What happens if you use the array index as a key?

9. What is the difference between a controlled and an uncontrolled input component?

Basic Theory

10. List the most important theoretical React interview topics a beginner should know (components, props, state, JSX, lifecycle, hooks).

11. What is the purpose of React Fragments (<>...</>)? Why not just wrap in a ?

12. What is conditional rendering in React? Show 2 different ways to implement it.

13. What is 'lifting state up' and when would you need to do it?

HTML/CSS in React context

14. What is the difference between inline vs block elements in HTML, and how does this affect layout inside React components?

15. How does className differ from class in JSX, and why does React require this?

MEDIUM LEVEL

Hooks in Depth

1. Explain useRef in depth. Name 2 use cases beyond referencing DOM nodes.

2. Explain useMemo and useCallback in detail — what problem do they solve, when should you actually use them, and what is the disadvantage of overusing them?

3. How do you build a custom hook? Write a custom useDebounce(value, delay) hook.

4. What is the purpose of the dependency array in useEffect and useCallback? What bugs occur from missing or incorrect dependencies?

Component Patterns

5. What are Higher Order Components (HOCs)? Give an example of a withAuth HOC.

6. What is React.memo (a Pure Component for functional components)? When does it NOT help performance?

7. Explain Context API vs Redux Toolkit in-depth, including caching/selector differences, boilerplate, devtools, and when to prefer one over the other.

8. What is prop drilling and what are 2-3 ways to avoid it?

Events & Rendering

9. Explain event propagation in React: bubbling and capturing. How do you stop propagation?

10. What is React's synthetic event system and why does it exist?

11. Explain React's batching behavior for setState calls. Does it differ inside event handlers vs setTimeout/promises pre-React 18 vs React 18?

12. What is the difference between setState in class components and the useState updater function in how they merge/replace state?

- ■ *setState merges the update with previous state (shallow merge); useState replaces the entire state slice*

Performance & Code-splitting

13. Explain React.lazy and Suspense in depth. How do they enable code-splitting?

14. How would you lazy-load images in a React app for performance?

15. What is the purpose of the useCallback hook specifically in preventing unnecessary child re-renders?

- ■ *Memoizes a function reference and prevents re-rendering of memoized child components*

16. How do you optimize asset/image loading and bundle size in a production React app?

Machine Coding — Practical

17. Implement a debounced search bar in React that calls an API only after the user stops typing for 300ms.

18. Implement pagination (with page numbers and next/prev) in React, fetching data from a backend API that supports page & limit query params.

19. Build a custom useFetch(url) hook that returns { data, loading, error } and handles cleanup on unmount (avoid setting state on an unmounted component).

20. Optimize this component — identify and fix all issues (missing key, missing dependency array causing infinite loop, direct mutation, no loading state):

```
function UserList({ users }) {
  const [filteredUsers, setFilteredUsers] = useState([]);
  useEffect(() => {
    setFilteredUsers(users.filter(user => user.active));
  });
  return (
    <div>
      {filteredUsers.map(user => (
        <div onClick={() => console.log(user.id)}>
          {user.name}
        </div>
      ))}
    </div>
  );
}
```

■ *Add [users] as dependency array, add a stable key={user.id}, add loading/empty checks, wrap the filter in useMemo instead of useEffect+state*

HARD LEVEL

React Internals

1. Explain React Fiber. What problem did it solve compared to the pre-React-16 stack reconciler?

2. Explain reconciliation in depth — how does React's diffing algorithm decide what to re-render, and what are the rules around keys that make list diffing efficient?

3. Explain the difference between the Virtual DOM and the Real DOM, and why diffing the virtual tree is cheaper than directly manipulating the real DOM.

4. What is React batching, and how did automatic batching change between React 17 and React 18 (including inside promises/setTimeout/native event handlers)?

5. What is the purpose of React's Concurrent Mode / Concurrent Rendering? How does it enable interruptible, non-blocking rendering?

Advanced Hooks & Patterns

6. Design and implement a custom hook (with machine-coding level detail) such as useLocalStorage, useOnClickOutside, or useInfiniteScroll — explain the cleanup and edge cases.

7. What does React's StrictMode do during development? Why might it cause effects/renders to fire twice?

8. Explain what the createPortal API does and give a real use case (e.g. modals, tooltips) where it's necessary.

9. What is a key benefit of Suspense for data fetching (not just lazy loading)? How does it differ from manual loading state management?

10. Design a generic HOC vs a custom hook for the same cross-cutting concern (e.g. authentication check) — discuss trade-offs (composability, hook rules, wrapper hell).

System-style / Architecture Questions

11. Design the file/folder structure for a large-scale, industry-level React.js codebase (feature-based vs type-based structuring, shared components, hooks, services, state layers).

12. Compare React.js vs Next.js in depth — SSR, SSG, ISR, routing, SEO implications, and when you'd choose one over the other.

13. How would you architect global state in a large React app: Context API, Redux Toolkit, Zustand, or React Query — what are you optimizing for in each choice?

14. How do you implement Role-Based Access Control (RBAC) on the frontend in React (route guarding, conditional UI, and why this must never be trusted as the only enforcement layer)?

Tricky Output-based / Debugging

15. Given a component that calls setState multiple times synchronously inside a single event handler, how many re-renders occur, and why?

16. Why can directly mutating an array/object in state (e.g. push()) fail to trigger a re-render even though the reference technically still points to the same array?

17. A useEffect with an async function directly as its callback throws a warning/anti-pattern — why is `useEffect(async () => {...})` incorrect, and what's the right pattern?

18. Explain a real production bug you'd expect from a stale closure inside useEffect/useCallback that doesn't include a changing variable in its dependency array.